

Flink State Backends 模块

核心流程与设计说明

基于 Apache Flink 1.15.4 Release 分支源码分析

flink-state-backends 5 子模块完整解读

涵盖: `flink-statebackend-rocksdb` / `forst` / `changelog` / `heap-spillable` / `common`

模块路径: `flink-state-backends/`

目录

- 一、模块概览与架构设计
- 二、RocksDB State Backend 核心类设计
- 三、ForSt State Backend 核心类设计
- 四、Changelog State Backend 核心类设计
- 五、Heap-Spillable 与 Common 子模块
- 六、Checkpoint / Snapshot 核心流程
- 七、恢复操作选择流程
- 八、ForSt 异步状态执行流程
- 九、Changelog 物化循环流程
- 十、CopyOnWrite SkipList 工作原理
- 十一、资源管理与配置体系
- 十二、关键设计模式与架构思想

一、模块概览与架构设计

1.1 模块定位

flink-state-backends 是 Apache Flink 的状态存储核心模块，提供了多种 KeyedStateBackend 实现方案。该模块决定了 Flink 有状态算子的状态如何存储、快照和恢复。包含 5 个子模块，覆盖了从嵌入式本地存储(RocksDB)到新一代异步存储(ForSt)、增量日志(Changelog)、可溢出堆存储(Heap-Spillable)以及公共物化接口(Common)的完整方案。

1.2 子模块一览

子模块	核心类数	定位与特点
flink-statebackend-rocksdb	79	基于RocksDB JNI嵌入式状态存储,支持增量Checkpoint
flink-statebackend-forst	108	下一代状态存储,支持异步IO/远程存储/State API v2
flink-statebackend-changelog	38	装饰器模式,记录状态变更日志,定期物化
flink-statebackend-heap-spillable	13	基于SkipList堆外内存状态存储,支持COW快照
flink-statebackend-common	2	公共接口:MaterializationTarget/Metadata

1.3 架构总览

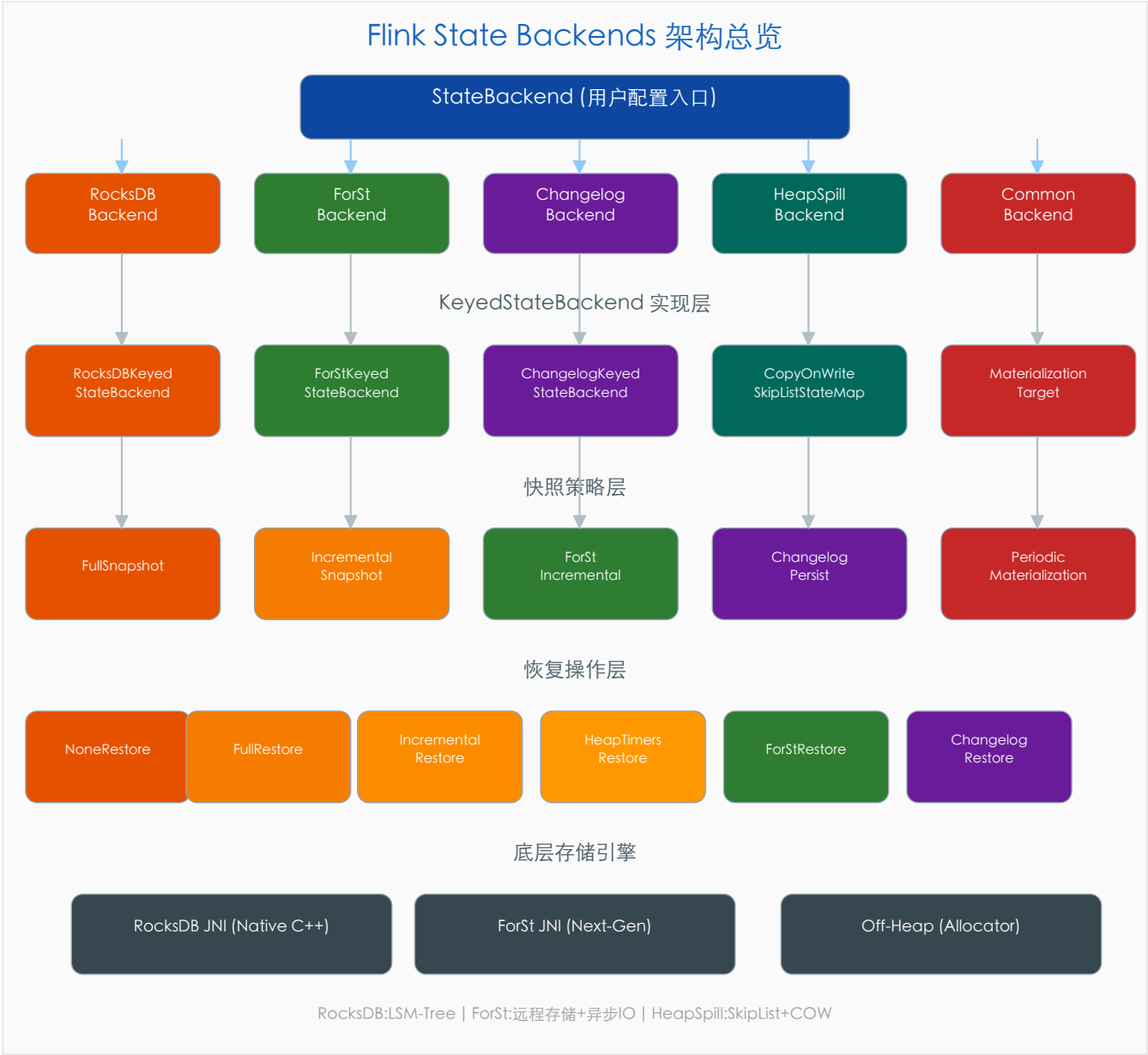


图 1-1: Flink State Backends 架构总览

二、RocksDB State Backend 核心类设计

2.1 EmbeddedRocksDBStateBackend (约1042行)

RocksDB State Backend 的工厂类和配置入口，实现 ConfigurableStateBackend 接口。负责创建 RocksDBKeyedStateBackend 实例，管理 JNI 库加载、本地目录初始化和共享内存资源分配。

- createKeyedStateBackend(): 核心工厂方法，编排整个初始化流程:
JNI加载->目录初始化->资源容器->Builder构建
- JNI库加载: 带重试的ensureRocksDBIsLoaded(), 支持自定义native库路径
- lazyInitializeForJob(): 惰性初始化本地目录，支持多目录轮转(instanceRocksDBPath数组)
- 共享内存管理:
allocateSharedCachesIfConfigured() 创建WriteBufferManager+BlockCache
- PredefinedOptions:
预定义模板(DEFAULT/SPINNING_DISK_OPTIMIZED/FLASH_SSD_OPTIMIZED等)
- RocksDBOptionsFactory: 用户自定义Options工厂，提供最高优先级配置覆盖

2.2 RocksDBKeyedStateBackend (约1152行)

实际的Keyed State Backend实现，将状态存储在嵌入式RocksDB实例中。每个状态注册为一个RocksDB Column Family。

- STATE_CREATE_FACTORIES: 静态Map映射StateDescriptor类型到对应创建工厂(策略模式)
- snapshot(): 委托给当前SnapshotStrategy(Full或Incremental)执行快照
- createOrUpdateInternalState(): 注册新状态或更新已有状态的Column Family，处理TTL
- RocksDBWriteBatchWrapper: 批量写入优化，积累到阈值再flush，减少JNI调用
- RocksDbTtlCompactFiltersManager: TTL过期状态通过RocksDB CompactFilter清理
- dispose(): 关闭所有Column Family Handle、释放RocksDB实例和原生资源

2.3 RocksDBKeyedStateBackendBuilder (约724行)

使用Builder模式组装RocksDBKeyedStateBackend，封装恢复和初始化逻辑。

- build(): 主流程—prepareDirectories->选择恢复操作->执行恢复->初始化快照策略->构建优先级队列工厂
- getRocksDBRestoreOperation():
根据StateHandle类型选择None/Incremental/Full/HeapTimersFull
- 恢复操作执行后获得RocksDBRestoreResult:
包含db实例、columnFamilyHandles、nativeMetricMonitor

2.4 RocksDBResourceContainer (约516行)

管理RocksDB原生Options对象完整生命周期，实现配置分层覆盖。确保JNI分配的原生内存在关闭时正确释放。

- 配置分层优先级: 用户Factory>Flink ConfigurableOptions>PredefinedOptions>RocksDB默认
- 共享BlockCache和WriteBufferManager: 所有ColumnFamily共享同一组缓存
- close(): 逐一释放所有已创建的Options对象，防止JNI内存泄漏

2.5 快照与恢复类

类名	核心职责
RocksNativeFullSnapshotStrategy	全量快照: getSnapshot()->遍历CF Iterator->写入CheckpointStream
RocksIncrementalSnapshotStrategy	增量快照: flush->getLiveFiles->diff->上传新增SST
RocksDBNoneRestoreOperation	创建空DB, 初始化默认ColumnFamilies
RocksDBFullRestoreOperation	反序列化全量快照数据, 逐KV写入新DB
RocksDBIncrementalRestoreOperation	下载SST+manifest, IngestExternalFile恢复
RocksDBHeapTimersFullRestoreOperation	全量恢复+Timer状态分离到Heap
RocksDBManualCompactionManager	手动Compaction管理, 优化SST文件布局
RocksDbTtlCompactFiltersManager	TTL CompactFilter, 通过RocksDB原生过滤器清理过期状态

三、ForSt State Backend 核心类设计

3.1 ForStStateBackend (约997行)

下一代状态存储后端(@Experimental)。基于ForSt(RocksDB增强分支)，支持远程存储溢出和异步状态操作(State API v2)。同时支持同步和异步两种模式。

- createAsyncKeyedStateBackend(): 创建ForStKeyedStateBackend, 返回AsyncKeyedStateBackend接口
- 远程存储: ForStFlinkFileSystem将checkpoint-dir作为远程溢出目标
- ForStPathContainer: 管理本地数据路径和远程路径映射
- 异步JNI加载: 使用executor在后台线程加载ForSt原生库

3.2 ForStKeyedStateBackend (约666行)

实现AsyncKeyedStateBackend<K>接口, 是State API v2核心实现。支持异步批量状态操作。

- createStateInternal(): 按状态类型(VALUE/LIST/MAP/REDUCING/AGGREGATING)创建实现
- createStateExecutor(): 创建ForStStateExecutor, 配置coordinator/read/write三个线程池
- dispose(): synchronized(lock)保护managedStateExecutors, 确保线程安全关闭

3.3 ForStStateExecutor (约343行)

异步批量状态请求执行器。Coordinator线程+Read线程池+Write线程池三层分离模型。

- executeBatchRequests(): 分类为Get/Iterate/Put三种请求
- ForStGeneralMultiGetOperation: 批量Get, 利用RocksDB multiGet API
- ForStWriteBatchOperation: 批量写入, WriteBatch原子写入
- callbackRunner: IO完成后回调Coordinator线程

3.4 其他关键类

类名	核心职责
ForStResourceContainer	管理ForSt Options, 配置分层逻辑与RocksDB版一致
ForStOperationUtils	ForSt操作工具: openDB()/createColumnFamily()
ForStFlinkFileSystem	桥接Flink FileSystem到ForSt Env, 支持远程存储
ForStIncrementalSnapshotStrategy	ForSt专属增量快照策略, 支持远程SST管理
ForStDBTtlCompactFiltersManager	ForSt专属TTL CompactFilter管理
ForStKeyedStateBackendBuilder	构建ForStKeyedStateBackend, 含异步IO线程池配置

四、Changelog State Backend 核心类设计

4.1 ChangelogStateBackend (约143行)

工厂类。使用装饰器模式在底层Backend(RocksDB/ForSt)之上添加Changelog日志层, 实现更小粒度Checkpoint。

- `restore()`:
编排恢复—`ChangelogBackendRestoreOperation`→创建`changelogWriter`→启动周期物化
- `DelegatedStateBackend`: 实际存储委托给配置的底层Backend
- `StateChangelogWriter`: 记录每个状态变更到Changelog

4.2 ChangelogKeyedStateBackend (约1201行)

装饰器类, 继承`AbstractKeyedStateBackend`并实现`MaterializationTarget`。拦截所有状态操作, 委托底层Backend同时记录变更。

- `snapshot()`: `stateChangelogWriter.persist(from,to)`持久化Changelog范围, 构建`ChangelogStateBackendHandle`
- `ChangelogSnapshotState`: 内部类, 封装已物化快照+未物化Changelog组合
- `initMaterialization()`: 触发底层Backend的snapshot作为物化操作
- 状态变更拦截:
每个`create/update/remove`通过`ChangelogState`包装器记录`StateChange`

4.3 PeriodicMaterializationManager (约427行)

周期性物化管理器, 调度定期将Changelog物化为底层Backend完整快照, 截断已物化Changelog段。

- `triggerMaterialization()`: 在`mailbox executor`中触发物化流程
- 随机初始延迟: `initialDelay×[0.75,1.25]`随机因子, 避免惊群效应
- 失败容忍: `allowedNumberOfFailures`配置, 超过后Task失败
- `asyncMaterializationPhase()`: 异步执行checkpoint数据上传

五、Heap-Spillable 与 Common 子模块

5.1 CopyOnWriteSkipListStateMap (约1589行)

基于SkipList的状态存储, 使用堆外内存和Copy-on-Write机制支持并发快照。

- SkipList多层索引: 随机层高, $O(\log n)$ 查找/插入/删除
- Copy-on-Write: `node.version < highestSnapshotVersion`时触发, 保留旧值供快照读取
- 逻辑删除vs物理删除: `remove`先标记, 快照完成后再物理删除回收空间
- Off-Heap内存: `Allocator+Chunk(1MB)+SpaceAllocator`, 避免GC压力
- `stateSnapshot()`: 递增版本号, 注册快照版本,
返回`CopyOnWriteSkipListStateMapSnapshot`

5.2 Common子模块

最小子模块(2个核心类), 为Changelog物化提供公共抽象。

- `MaterializationTarget`: 物化目标接口, 被`ChangelogKeyedStateBackend`实现
- `ChangelogMaterializationMetadata`: 封装物化操作元数据(ID/起始序列号/目标序列号)

六、Checkpoint / Snapshot 核心流程

Checkpoint是Flink容错核心。State

Backend负责将算子状态持久化到分布式存储。RocksDB支持全量和增量两种快照策略。

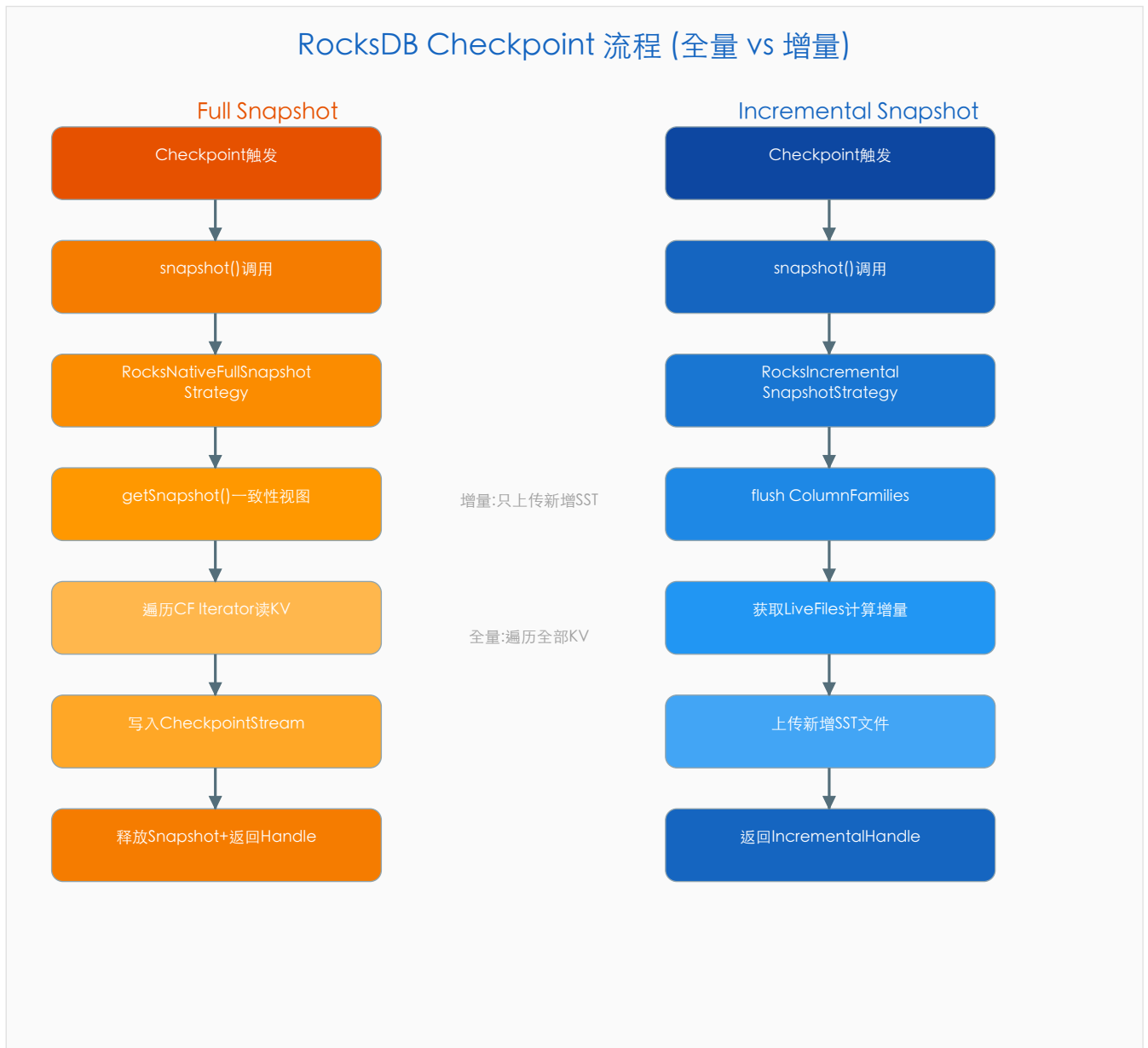


图6-1: RocksDB Checkpoint流程(全量 vs 增量)

- 全量快照: 使用RocksDB原生Snapshot获取一致性视图, 遍历所有ColumnFamily Iterator读取全部KV
- 增量快照: flush memtable后获取LiveFiles列表, 对比上次checkpoint计算新增SST文件, 仅上传新增SST
- 异步执行: 同步阶段(获取Snapshot/LiveFiles)在主线程, 异步阶段(数据上传)在独立线程池
- KeyGroupRange分组: 数据按KeyGroup范围分区写入, 支持rescale时细粒度状态重分配

七、恢复操作选择流程

RocksDB Backend恢复时根据StateHandle类型选择合适的恢复操作。

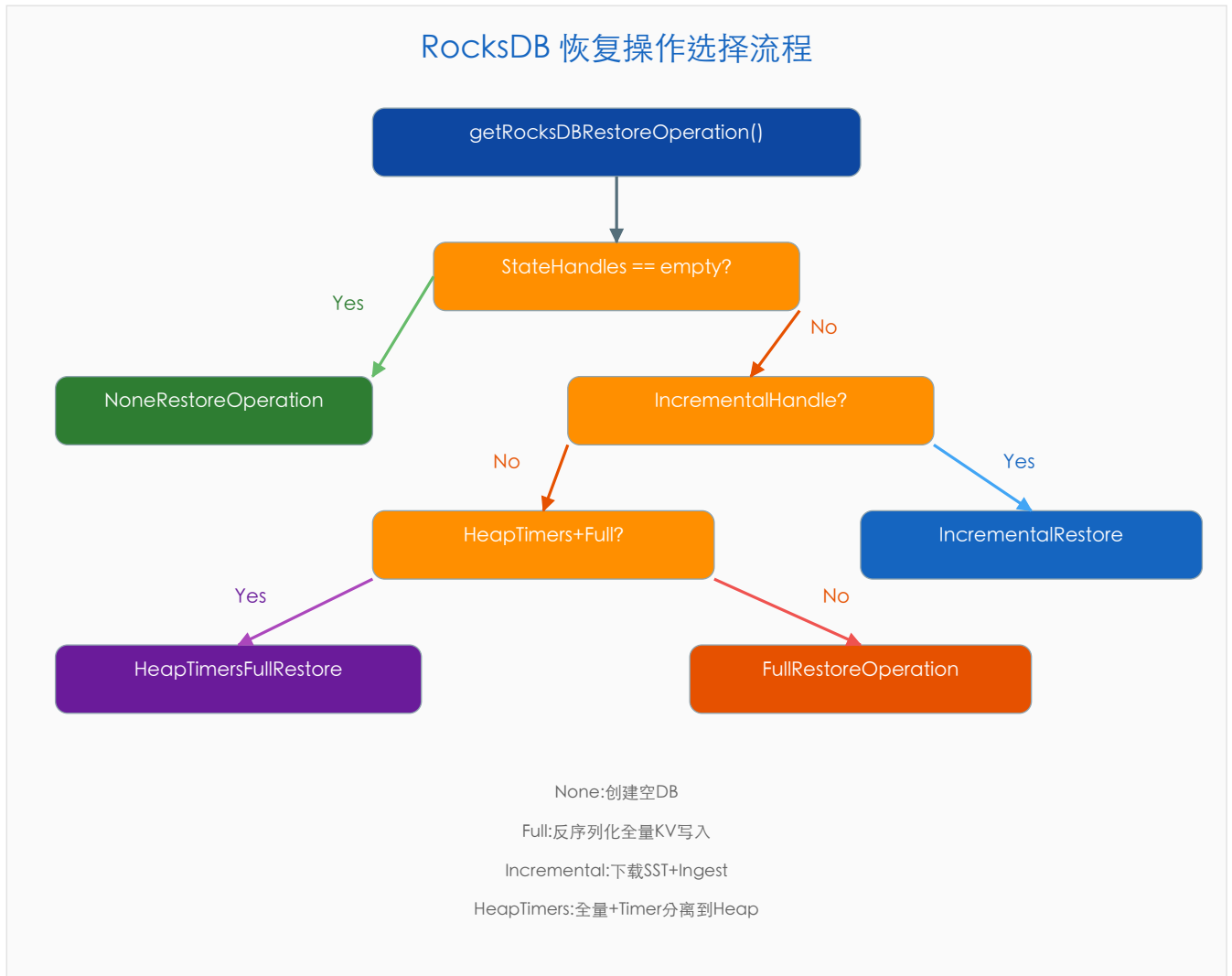


图7-1: RocksDB恢复操作选择流程

- NoneRestore: 全新启动, 创建空RocksDB实例
- FullRestore: 反序列化CheckpointStream中KV数据, 逐条写入新RocksDB
- IncrementalRestore: 下载SST文件和MANIFEST, 使用IngestExternalFile恢复
- HeapTimersFullRestore: 全量恢复变种, Timer状态分离到Heap
- Changelog恢复: 先恢复物化快照, 再回放Changelog增量日志

八、ForSt 异步状态执行流程

ForSt引入异步批量状态执行模型(State

API

v2)。ForStStateExecutor将请求分类后分发到独立Read/Write IO线程池, 实现读写分离。

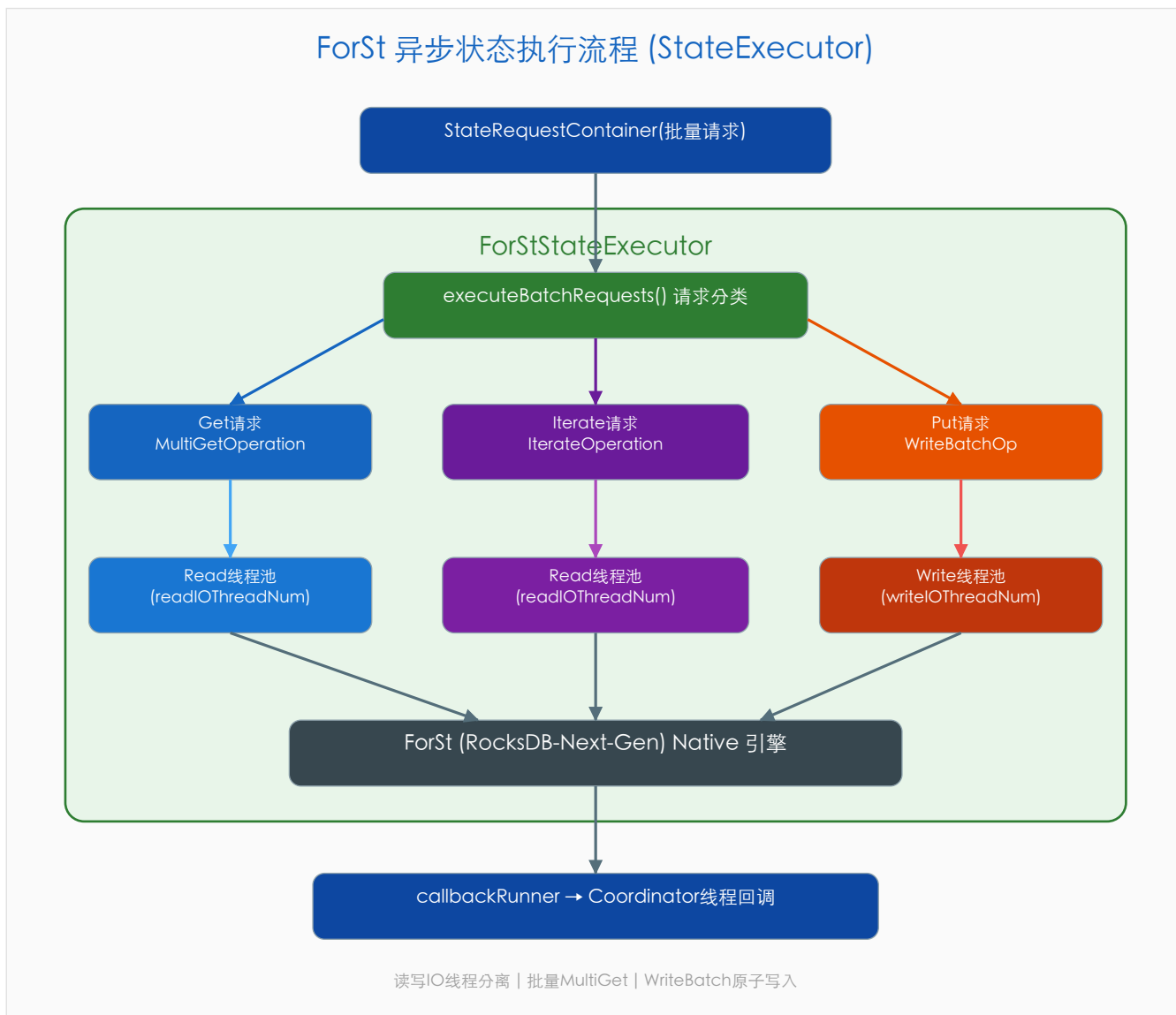


图8-1: ForSt异步状态执行流程

- 读写线程分离: Read线程处理Get/Iterate, Write线程处理Put, 避免相互阻塞
- 批量MultiGet: 多个Get请求合并为RocksDB multiGet API调用
- WriteBatch原子写入: 多个Put合并为WriteBatch, 减少WAL写入次数
- Coordinator线程负责请求分类和结果收集, 不执行IO操作

九、Changelog 物化循环流程

Changelog通过定期物化(Periodic Materialization)控制日志增长。物化将Changelog持久化为底层Backend完整快照,截断已物化Changelog段。

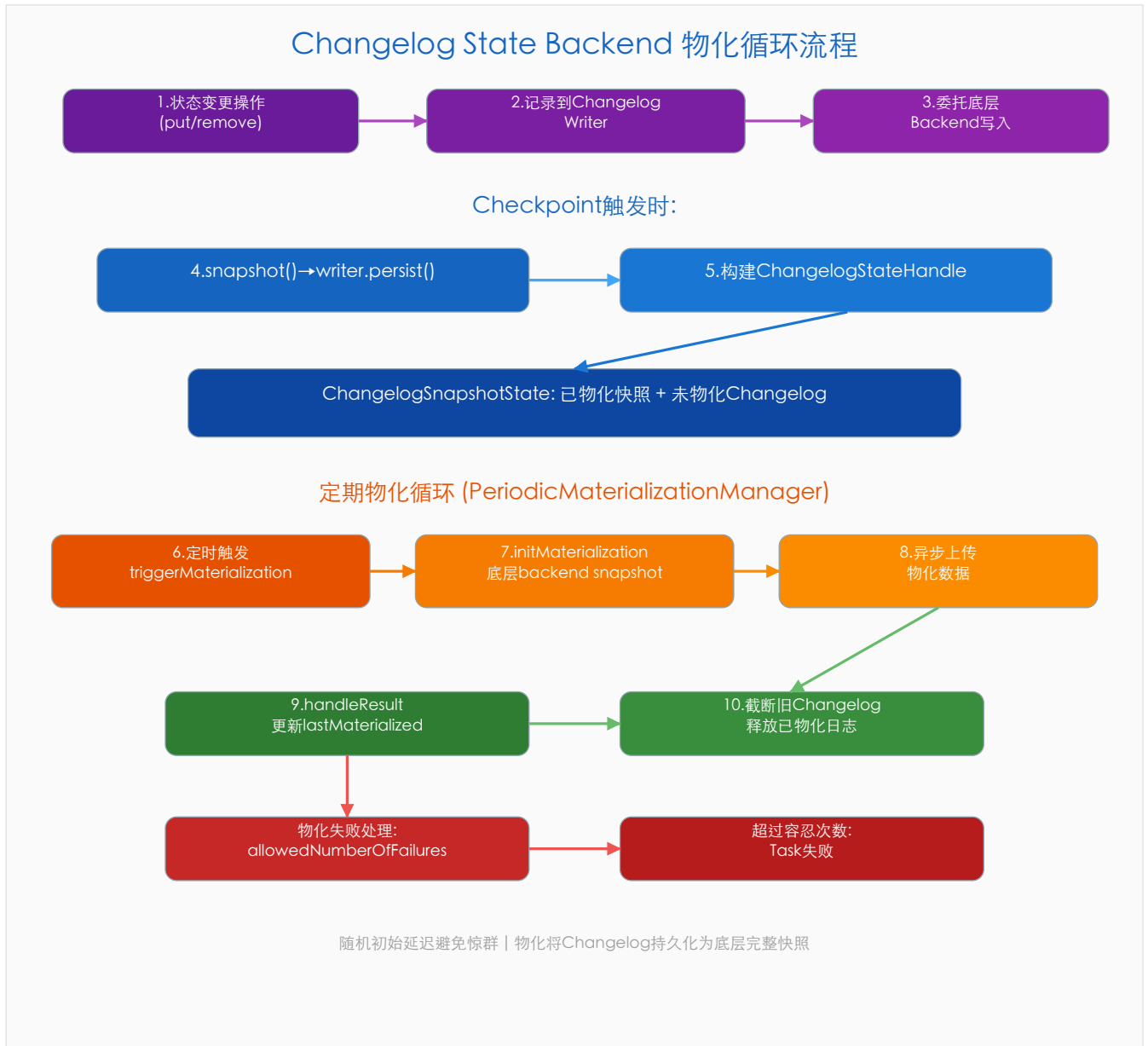


图9-1: Changelog物化循环流程

- Checkpoint时持久化Changelog范围[`lastMaterialized`,`current`], 物化后移动`lastMaterialized`
- 惊群效应防护: 初始延迟×随机因子[0.75,1.25]
- Mailbox线程模型: `triggerMaterialization`在mailbox线程中执行, 确保线程安全
- 失败容忍: `allowedNumberOfFailures`, 超过后Task失败

十、CopyOnWrite SkipList 工作原理

CopyOnWriteSkipListStateMap结合SkipList有序索引和COW并发快照支持，使用堆外内存减少GC开销。

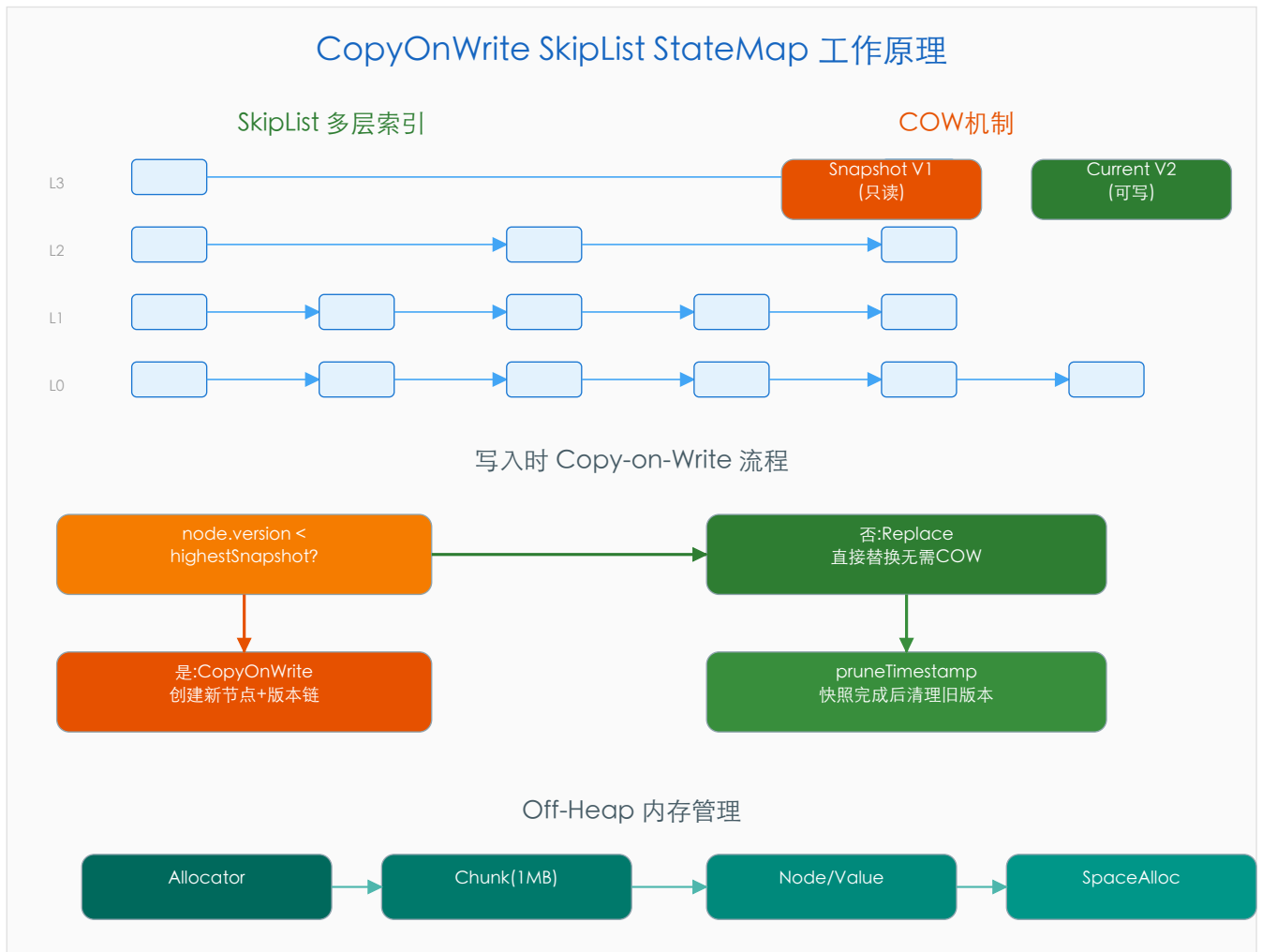


图10-1: CopyOnWrite SkipList工作原理

- Off-Heap存储: 所有Key/Value在堆外内存中, 通过long地址引用
- 版本链: Value节点有nextVersion指针, 快照读取时沿版本链查找匹配版本
- COW条件: 仅node.version < highestSnapshotVersion时触发, 最小化复制开销
- 空间回收: pruneTimestamp/pruneSpace在快照完成后清理旧版本, 回收堆外内存

十一、资源管理与配置体系

RocksDB和ForSt的配置体系采用分层覆盖设计，通过ResourceContainer统一管理Options生命周期。



图 11-1: RocksDB资源容器配置层次

配置分类	关键配置项	说明
内存管理	state.backend.rocksdb.memory.managed	共享内存分配比例
写入优化	writebuffer.size/count	Write Buffer参数
Compaction	compaction.style/level.target-file-size	Compaction策略
读取优化	block.blocksize/cache-size/bloom-filter	Block Cache和布隆过滤器
增量CP	state.backend.incremental	增量Checkpoint配置
TTL	ttl.compaction.filter.enabled	TTL CompactFilter开关
Timer	timer-service.factory	Timer存储(ROCKSDB/HEAP)
Changelog	periodic-materialization-interval	物化间隔和失败容忍
ForSt IO	read/write.io-thread-num	ForSt读写线程数

十二、关键设计模式与架构思想

Builder 模式

RocksDBKeyedStateBackendBuilder/ForStKeyedStateBackendBuilder —
封装复杂的恢复、快照策略初始化和资源管理逻辑, 通过build()方法一步完成Backend构建

装饰器模式

ChangelogKeyedStateBackend包装底层RocksDB/ForSt Backend,
在不修改底层实现的情况下添加Changelog记录功能

策略模式

SnapshotStrategy接口 (Full/Incremental),
RestoreOperation接口 (None/Full/Incremental/HeapTimers) —
运行时根据配置和状态选择具体策略

工厂模式

STATE_CREATE_FACTORIES映射StateDescriptor到创建工厂;
StateBackend.createKeyedStateBackend()工厂方法

Copy-on-Write

CopyOnWriteSkipListStateMap在有活跃快照时创建新版本节点, 旧版本通过版本链保留,
实现无锁并发快照

资源容器

RocksDBResourceContainer/ForStResourceContainer — 统一管理JNI原生Options生命周期,
防止内存泄漏

配置分层覆盖

四层配置优先级: 用户Factory>ConfigurableOptions>PredefinedOptions>Default,
灵活性和安全性兼顾

读写分离

ForStStateExecutor将IO操作分为Read线程池和Write线程池, 避免读写相互阻塞, 提升并发吞吐

定期物化

PeriodicMaterializationManager定时将Changelog物化为底层完整快照, 随机延迟避免惊群,
失败容忍机制保证鲁棒性

异步CompletableFuture编排

恢复操作、快照上传、物化等长耗时操作均使用CompletableFuture异步执行, 不阻塞主处理线程